

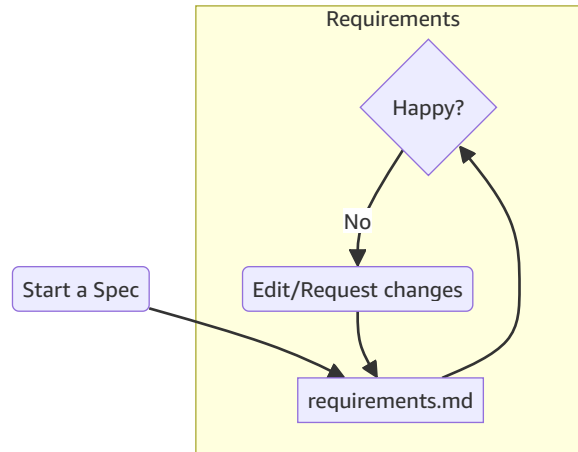


Generate requirement specifications

Overview

In this section you will start to write specifications for your Flappy Kiro game, starting with the initial requirements.

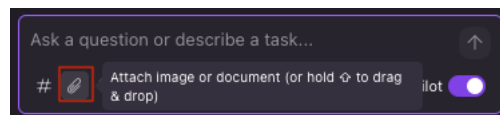
You will use the **spec mode** to generate the [Kiro requirement](#) file. These requirements are written in the form of user stories with acceptance criteria in [Easy Approach to Requirements Syntax](#) (EARS) notation.



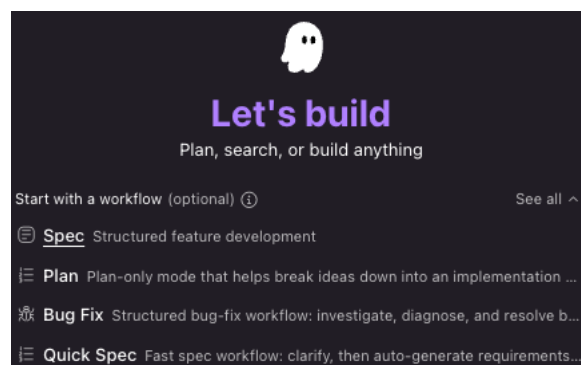
Create application specifications

In the **Kiro chat panel**, you will use Kiro to create the specification for your Flappy Kiro game.

- You can use natural language to describe your target program.
- You can give additional context, such as pointing Kiro to an existing file or folder, by using the # key.
- You can even add images to the prompt by choosing the **Attach image or document** icon!



1. Select **Spec**.



2. Prompt Kiro with some basic information about your Flappy Kiro game. To include a mock-up of the user-interface in your prompt, choose **Add images** icon, and select the `img/example-ui.png` file.

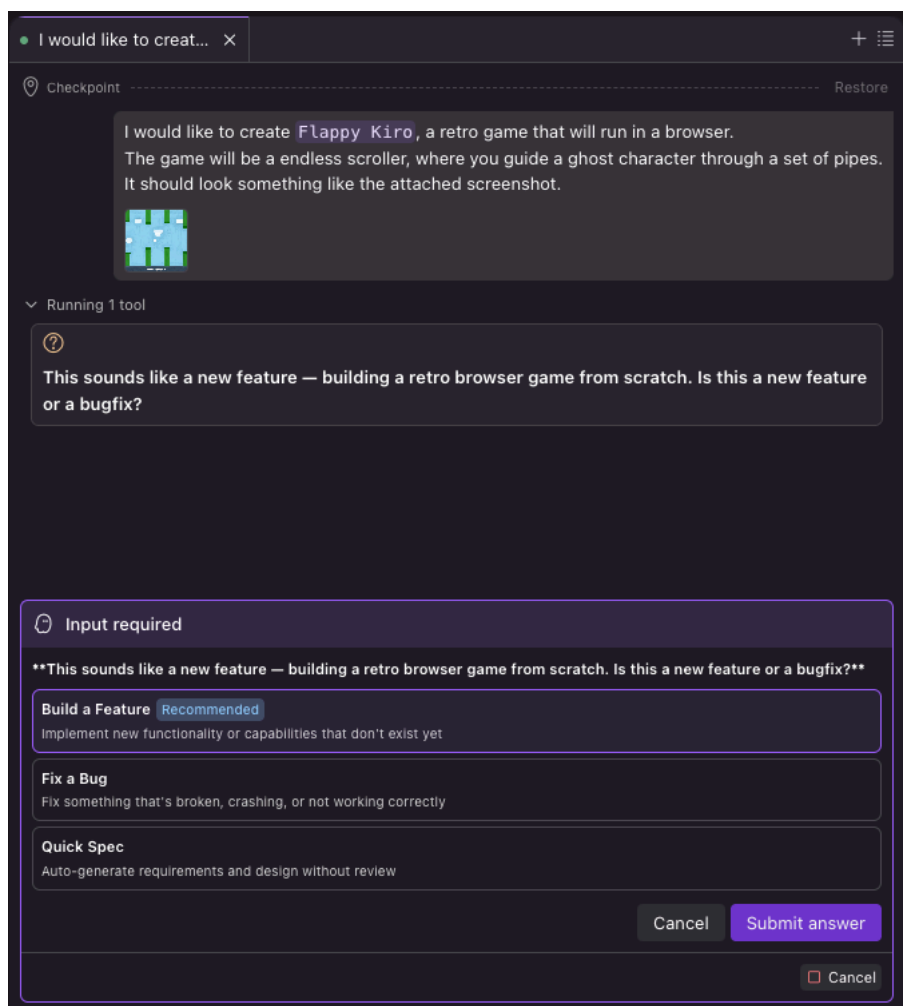
An example prompt may be:

```
I would like to create `Flappy Kiro`, a retro game that will run in a browser.  
The game will be a endless scroller, where you guide a ghost character through a set of pipes.
```



It should look something like the attached screenshot.

3. When you are happy with your prompt, hit **Enter** to send the prompt to Kiro.
4. As this a new application, ensure that **Build a Feature** is selected, and then choose **Submit answer**.

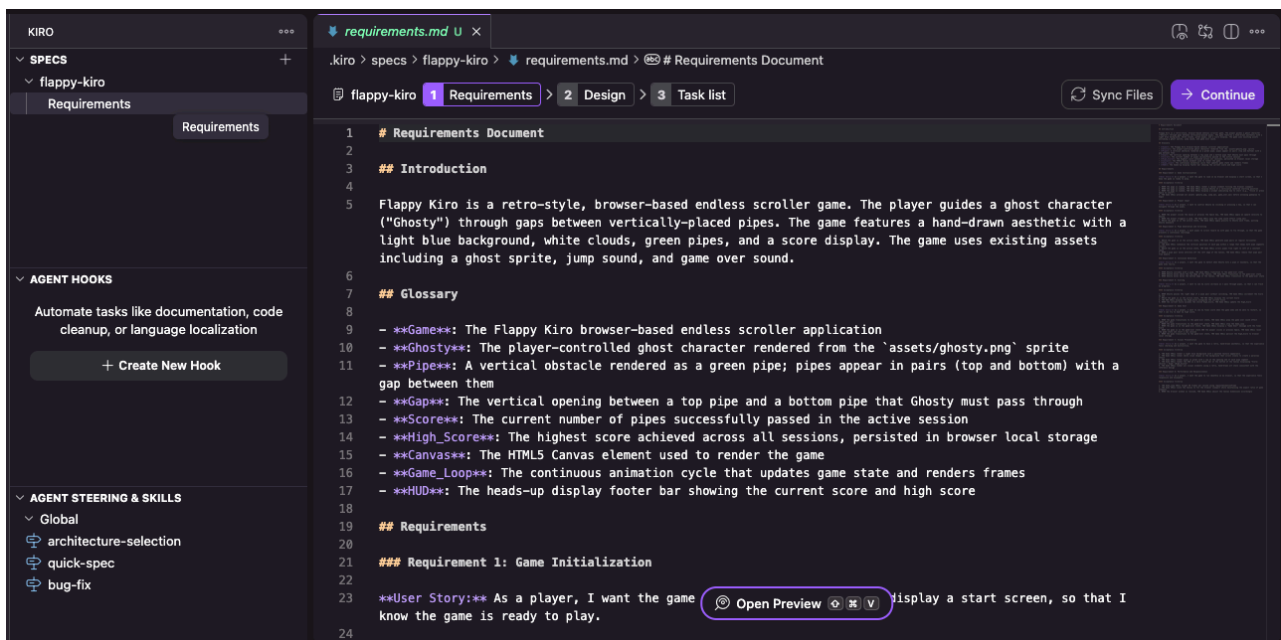


5. When asked what do you want to start with?, select **Requirements**, and then choose **Submit answer**.

After a few moments, Kiro will generate a `requirements.md` file creating draft requirements. A summary of key elements are also shown in the Kiro chat panel.

Examine the draft requirements

The `requirements.md` document is located in the `.kiro/specs` folder. By selecting the **Kiro** icon in the view panel, you will see the requirements displayed in the **Specs** pane in the **Kiro panel**.



Refine the requirements

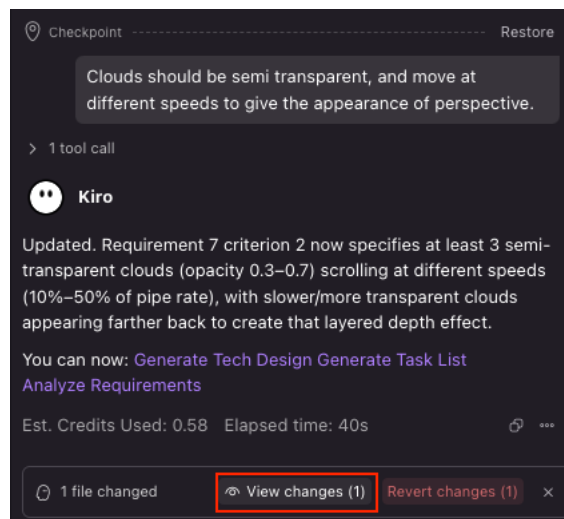
After reviewing the specification, you can ask KIRO to enhance the specifications.

Update the clouds

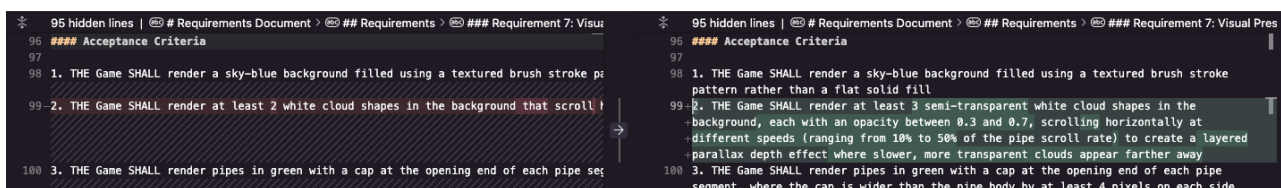
1. Write a prompt to update how clouds appear.

► Example prompt

KIRO will update `requirements.md`, and summarize the changes in the chat panel. To see the update, select **View changes (1)**.



This will show the updated files, highlighting the changes with deletions in red and additions in green.



You can undo updates using **Revert changes (1)**.

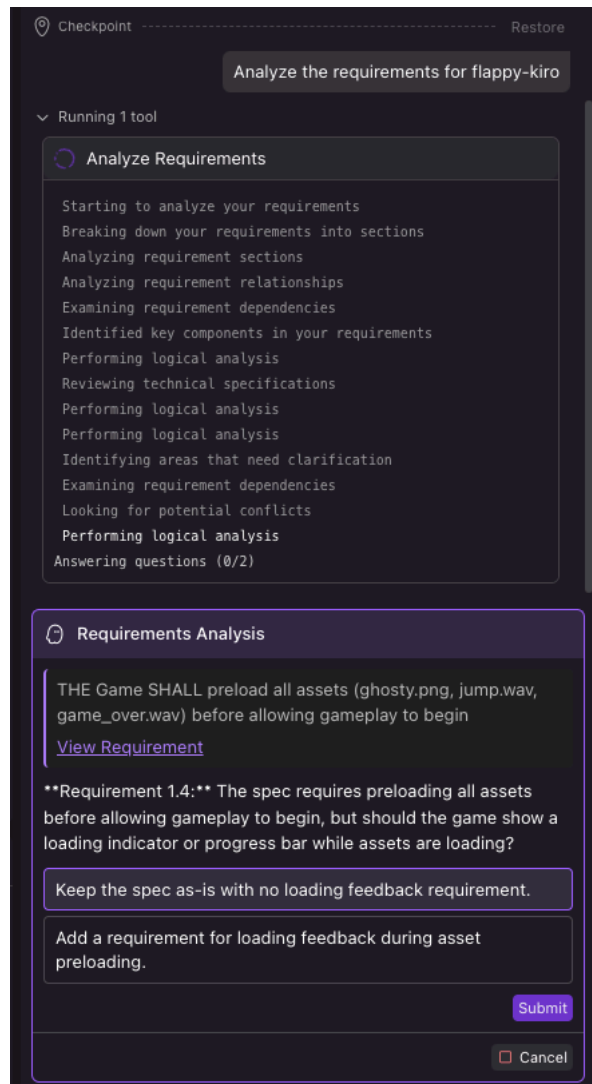
2. Continue to refine the requirements until you are happy with the overall specification.

► Example prompts

(Optional) Analyze the requirements

Before moving to design, ask Kiro to analyze your requirements for inconsistencies, ambiguities, and gaps. Kiro reasons across the whole requirement set - not each item in isolation - to catch issues that are easy to miss when reading through. See [Analyze Requirements](#) in the Kiro documentation for more information.

1. Ask Kiro to do a deep analysis by selecting **Analyze Requirements**.



2. For each item identified, choose to either keep the existing specification, or extend the spec to include the proposed change.

Complete the requirements

1. Make a commit of requirements to your local Git repository with the following command in your IDE terminal window:

```
git add .
git commit -m 'Produced requirements'
```



2. Tell Kiro to move to the design phase by selecting **Generate Tech Design**.

The requirements document for Flappy Kiro is ready. It covers 8 areas: game initialization, player input/physics, pipe generation, collision detection, scoring with persistence, game over flow, the retro visual style, and performance targets — all with precise, testable acceptance criteria.

Take a look and let me know if you'd like to adjust anything.

You can now: [Generate Tech Design](#) [Generate Task List](#) [Analyze Requirements](#)

Est. Credits Used: 5.96 Elapsed time: 10m 41s

Wrap up

You have completed the requirements document.

Next you will produce, review, iterate and complete the detailed system design.

[Previous](#)[Next](#)